

RAILTIME

Software-Deployment & Geräte-Provisionierung

Technische Planungsgrundlage

Planungsstand: 19. August 2026

Repository: lucasz1991/App

Ablage: .dev/railtime-software-deployment-plan/

Status: Planungsgrundlage - keine Implementierungsfreigabe

Dieses Dokument hält den aktuellen Plan für zentral verwaltete Softwarebereitstellung auf RailTime-Geräten fest. Die ausführlich editierbare Fassung liegt als Markdown-Datei im selben Ordner.

1. Ziel

RailTime steuert zentral, welche Software auf verwalteten Geräten installiert sein soll - bei der Ersteinrichtung und später für Nachinstallationen, Updates und Deinstallationen.

Die App bleibt fachliche Control Plane für Sollzustand, Freigaben, Berechtigungen, Audit und Deployments. Die technische Installation wird capability-basiert an Provider bzw. Device-Agent delegiert.

2. Vorhandene Basis

Bereits vorhanden: Device, DeviceCommand, DeviceProvisioningProfile, DeviceArtifact, DeviceCommandService, DeviceProviderRegistry, DeviceManagement und die devices-Queue.

DeviceCommandType enthält bereits `install_software`, `uninstall_software` und `apply_profile`. Deshalb wird keine zweite Remote-Command-Architektur aufgebaut.

3. Neue Bausteine

Unter Geräte & Lager kommen Software-Katalog, Software-Profil und Deployments hinzu. Gerätedetails erhalten einen Tab Software mit Soll, Ist, Version, Zuweisungsquelle, laufendem Deployment und Fehlerstatus.

Der Software-Katalog beschreibt Paketname, Plattform, Quelle (z.B. winget, MSI, EXE, PKG, DMG, APK, Store/MDM), Package-ID, Zielversion, optional DeviceArtifact, Install-/Uninstall-Parameter und Erkennungsmethode.

4. Soll-/Ist-Modell

Der Sollzustand wird strikt vom belegbaren Istzustand getrennt. Ein gesendeter Command bedeutet nicht, dass die Software installiert ist. Completed sollte erst nach Provider-/Agent-Verifikation des Zielzustands gelten.

Empfohlene Tabellen: `device_software_packages`, `device_software_profile_items`, `device_software_assignments`, `device_installed_software`; optional `device_software_deployments` als fachlicher Kopf für Massenaktionen.

5. Erst-Provisionierung

Ablauf: Gerät erfassen -> Mitarbeiter/Gruppe zuordnen -> Enrollment -> Softwareprofil wählen -> Istinventar empfangen -> Soll/Ist vergleichen -> fehlende Pflichtsoftware planen -> `install_software` Commands -> Provider/Agent -> Verifikation -> Readiness aktualisieren.

Beispiel Standard Arbeitsplatz: Chrome, Microsoft 365, Outlook, Teams, OneDrive, Adobe Reader, 7-Zip und RailTime-Komponenten.

6. Spätere Deployments

Dieselbe Pipeline wird für Einzelgeräte, mehrere Geräte, Gerätegruppen und Profile verwendet. Offline-Geräte behalten offene Aufträge. Massenaktionen erzeugen pro Gerät nachvollziehbare Commands, damit Teilfehler sichtbar bleiben.

Deinstallation erfolgt über `desired_state=absent` und `uninstall_software`, sofern Provider und Paket dies unterstützen. Kritische Pakete können geschützt werden.

7. Sicherheit und Provider

Keine frei editierbaren Shell-Befehle als normale Pakete. Eigene Installer nur als freigegebene DeviceArtifact-Version mit SHA-256. Keine Credentials oder Tokens in DeviceCommand.payload. Der globale Kill-Switch und Provider-Capabilities bleiben bindend.

Für einen eigenen Windows-Agenten ist ein Windows Service zu bevorzugen, damit Installationen unabhängig von Benutzer-UI, Login und Neustarts laufen.

8. Umsetzung

Phase 1: Datenmodell, Katalog, Policies und Artifact-Verknüpfung. Phase 2: Einzelgerät-Deployment über vorhandenen DeviceCommandService. Phase 3: Softwareprofil bei Enrollment plus Reconciliation und Readiness. Phase 4: Gruppen-/Massenrollout. Phase 5: Versionen, Updates und Compliance.

Erster technischer Durchstich: Software-Katalog -> Gerätezuweisung -> install_software -> Provider/Agent
-> Ist-Erkennung -> RailTime-Status.

9. Offene Entscheidungen

Vor Implementierung festlegen: Profiltrennung, tatsächliche Provider-Inventarfähigkeiten,
Windows-Provider/Agent, Standardsoftware, Self-Service, Lizenzmodell, Updatepolitik, geschützte Pakete,
Wartungsfenster und Einfluss auf Readiness.